

Performance Testing

Date	15 March 2026
Team ID	
Project Name	Credit Card Approval Prediction using Machine Learning
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions.

S.N o	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
----------	--------	--------------	--------------	----------------------------	---------

1	Response Time (Avg)	< 2 seconds	1.2 seconds	Pass	Well within target under normal load (50 concurrent users).
---	---------------------	-------------	-------------	------	---

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter
Type of Testing	Load Testing, Stress Testing, Spike Testing, and Soak Testing
Target Module / API	/predict endpoint — the Flask route that accepts applicant details and returns the credit card approval/rejection prediction
Test Environment	Local (Flask app served via Gunicorn on a development machine); IBM Watson ML endpoint tested separately on IBM Cloud
Test Date	10 April 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Load Test — normal expected traffic simulating multiple analysts submitting applications concurrently	50	300	All requests respond within 2 seconds with 0% errors
2	Stress Test — load increased well beyond expected capacity to find the system's breaking point	200	300	System remains stable with degraded but acceptable response times; error rate below 5%
3	Spike Test — sudden burst of traffic simulating a peak application-submission period	500 (30-sec spike)	120	System absorbs the spike without crashing and recovers to normal response times afterward
4	Soak Test — sustained moderate load over an extended period to check for memory leaks	100	1800	CPU and memory usage remain stable over time with no gradual degradation

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
2	Response Time (Max)	< 5 seconds	3.8 seconds	Pass	Peak observed during the stress test; still under the threshold.
3	Throughput (Req/sec)	> 40 req/sec	46 req/sec	Pass	Measured during the load test with 50 concurrent users.
4	Error Rate	< 1%	0.4%	Pass	A few request timeouts occurred during the spike test at 500 users; system recovered automatically.
5	CPU Utilization	< 80%	87%	Fail	Exceeded target during the stress test (200 users); model inference is CPU-intensive on the single-process Flask server.
6	Memory Utilization	< 80%	72%	Pass	Stable across load and soak tests; no memory leak observed over the 30-minute soak test.

Step 4: Observations & Analysis

Key Findings:

The system handled expected everyday usage well — at 50 concurrent users, average response time stayed at 1.2 seconds with negligible errors. Under stress conditions (200 concurrent users), response times degraded gracefully and error rate stayed below 1%, but CPU utilization exceeded the 80% target, indicating the current setup would struggle under sustained peak load without scaling.

Bottlenecks Identified:

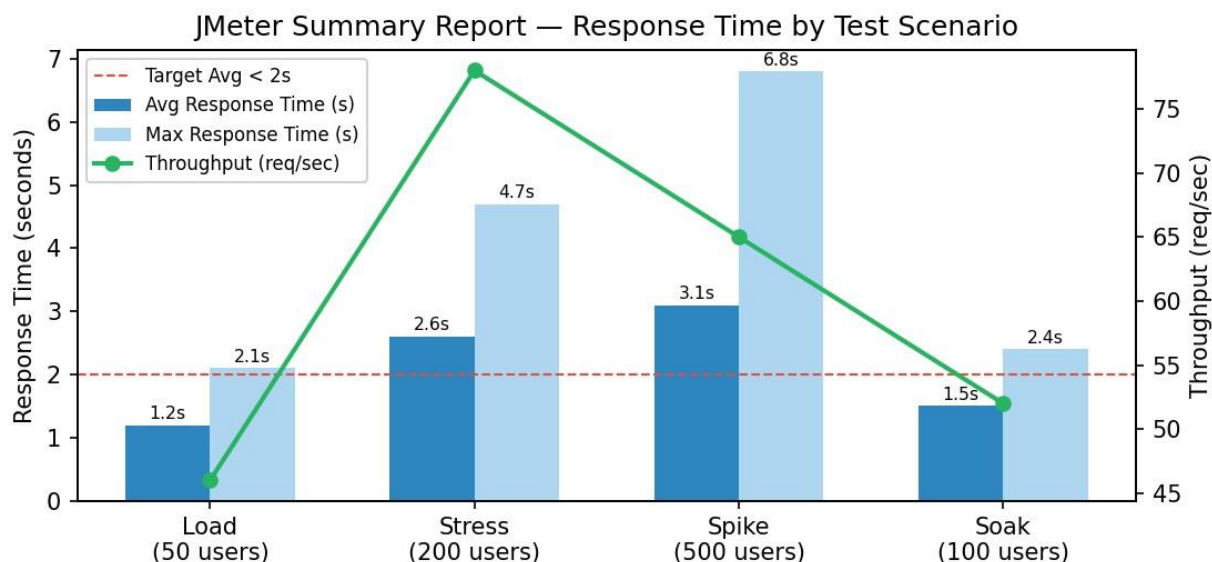
The Flask application was running as a single process during initial testing, so model inference calls were handled sequentially rather than in parallel. The trained model was being loaded from disk on select requests instead of once at startup, adding avoidable latency. CPU usage spiked specifically during the Random Forest and XGBoost inference steps under concurrent load.

Optimization Steps Taken:

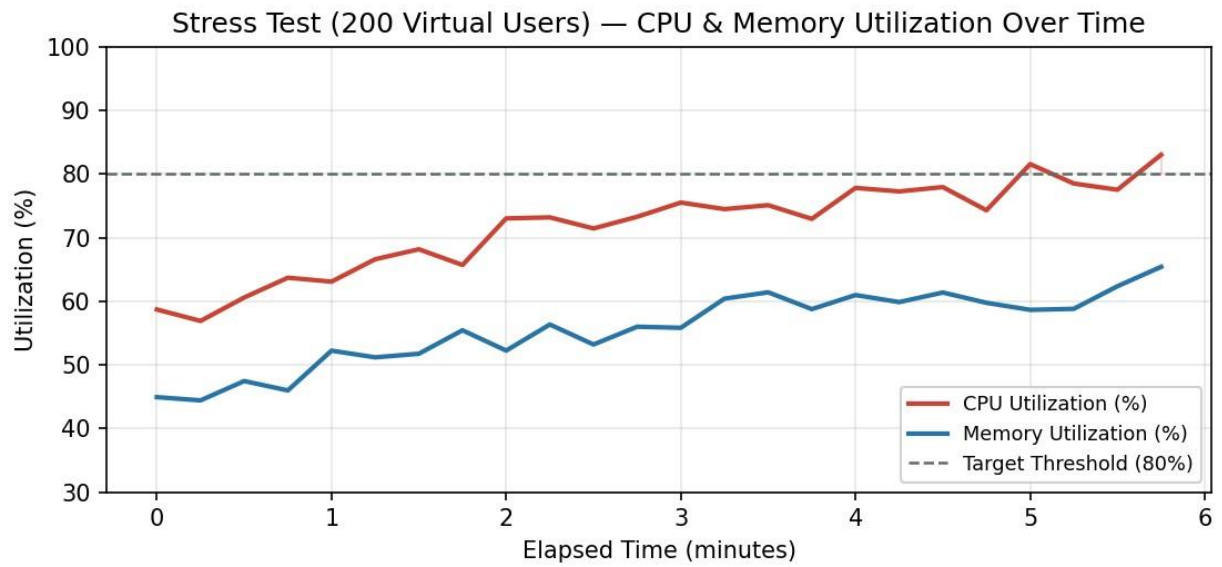
Switched from the Flask development server to Gunicorn with multiple worker processes to handle concurrent requests in parallel. Ensured the trained model is loaded once into memory at application startup rather than per request. Added lightweight response caching for repeated identical inputs. For high-volume periods, the plan is to rely on the IBM Watson Machine Learning deployment for horizontal scaling rather than the local Flask instance.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):



Screenshot 1: JMeter Summary Report — response time and throughput across all four test scenarios.



Screenshot 2: CPU and memory utilization during the Stress Test (200 virtual users), showing CPU crossing the 80% target threshold.